

2D 工程图纸 → 3D 模型：自然过渡是怎么做出来的？

从一条推文出发：开发者 Dilum Sanjaya 用 Kimi K3 做了一个应用——2D 示意图自然地“长”成 3D 坦克（坦克完全由代码生成，还能一键换成机器人）。本指南用 Three.js 手工复刻同款效果，把背后的每个机制讲透。

来源：<https://x.com/DilumSanjaya/status/2092293934844346660> | 复刻实验：[blueprint-to-3d/exercise/](#) | 2026-10-02

疑问清单（疑问即目录）

1. **Q1** 这条推文里的应用到底展示了什么？
2. **Q2** “坦克完全由代码生成”是什么意思？
3. **Q3** “自然过渡到 3D”视觉上是怎么做到的？
4. **Q4** 补间动画与缓动函数是什么？（新概念）
5. **Q5** 图纸线条“重绘”动画的原理？
6. **Q6** 图纸为什么也要用代码生成？
7. **Q7** 尺寸标注线（工程图味儿）怎么画？
8. **Q8** 炮塔转动时炮管为什么自动跟着转？
9. **Q9** “可替换为其他模型”架构上怎么设计？
10. **Q10** 为什么必须用 http 打开、双击 html 不行？
11. **Q11** Kimi K3 在这类 vibe coding 里干什么？
12. **Q12** 没有显示器也能验证 3D 页面跑通了？

1 这条推文里的应用到底展示了什么？

开发者 **Dilum Sanjaya**（Senior 工程师，以“vibe coding”交互式 3D 小应用著称）用 **Kimi K3**（月之暗面的旗舰代码模型）做了一个网页应用：**左侧是一张 2D 工程示意图（坦克侧视图），它自然地“过渡”成一个可旋转交互的 3D 坦克**。推文原话：“Exploring how 2D schematics can transition naturally into 3D. Made this app with Kimi K3. The tank is fully generated with code. Super easy to swap it out for something else.”

三个要点：① **过渡要“自然”**（不是硬切换，而是图纸线条与立体零件——对应地长出来）；② **模型零素材**（没有 .glb/.obj 文件，坦克是代码现场“捏”的）；③ **模型可替换**（换一个 builder 函数就换一台机器，他还在尝试让 AI 直接生成准确的机器人设计图纸）。

作者没有开源代码，所以本指南的复刻实验（见文末）是**按推文描述的效果用 Three.js 从零手工实现的**——这也正好是把每个机制拆开学的最好方式。

2 "坦克完全由代码生成"是什么意思？

这正是知识库里有概念 **程序化建模**（见 vault 概念笔记）：传统做法像"买现成泥人"——在 Blender 里捏好、导出模型文件、网页加载；程序化建模像"写菜谱"——代码里写明"车体是 $3.6 \times 0.9 \times 1.6$ 的盒子、炮塔是八棱柱、炮管是细圆柱"，运行时执行菜谱当场造出坦克。

加载模型文件（泥人成品）

Blender 手工捏 → 导出 .glb → 网页下载加载
文件可能几 MB；改形状要回建模软件

程序化建模（菜谱）

`BoxGeometry(3.6, 0.9, 1.6) = 车体`
代码即模型，改参数整车自动跟着变

图 1：加载文件 vs 程序化生成——本复刻的坦克 14 个零件全部用 Three.js 几何体 API 现场创建

对"图纸→3D"这个场景，程序化建模还有一个独家优势：**图纸上的每一根线条都能对应到代码里的一个零件**，过渡动画才可以"逐件"发生（Q3）。

3 "自然过渡到 3D"视觉上是怎么做到的？

复刻版把"自然"拆成两个同时发生的动画：

第一步：图纸线条重绘。左侧 SVG 图纸的所有轮廓线从"空白"逐条描出来（像有人在现场画图），尺寸标注随后浮现，图纸整体淡出到 25% 透明度退居二线。

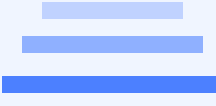
第二步：零件逐件"挤出"。右侧 3D 场景里，每个零件初始被压扁成一张纸片（y 方向缩放 0.02），然后按 **90 毫秒错峰（stagger）**逐件"长"回立体——用的还是带回弹的缓动（Q4），像果冻一样抖一下定型。因为零件顺序就是图纸从上到下的顺序，观众的眼睛自然把"图纸线"和"立体件"——配对。

① 图纸重绘



stroke-dashoffset 递减

② 零件挤出



错峰 90ms · 由扁到立

③ 开放交互



可拖动旋转 / 滚轮缩放

图 2：过渡的三幕剧——重绘图纸 → 逐件挤出 → 开放交互（动画结束后才启用鼠标控制）

4 补间动画与缓动函数是什么？（本批新概念）

补间 (tween) = 你只指定**起点、终点、时长**，中间每一帧的值由公式自动算出。本复刻里每个零件的"挤出"就是一个补间：起点 $\text{scale.y}=0.02$ ，终点 $\text{scale.y}=1$ ，时长 700 毫秒。

决定"中间值怎么变"的公式叫**缓动函数 (easing)**。同样 700 毫秒，线性缓动像电梯匀速升降；

easeOutBack 则**先冲过头再回弹**，像捏橡皮泥——本复刻用它就是为了让零件"长出来"带一点 Q 弹，这也是"自然"二字的来源之一。

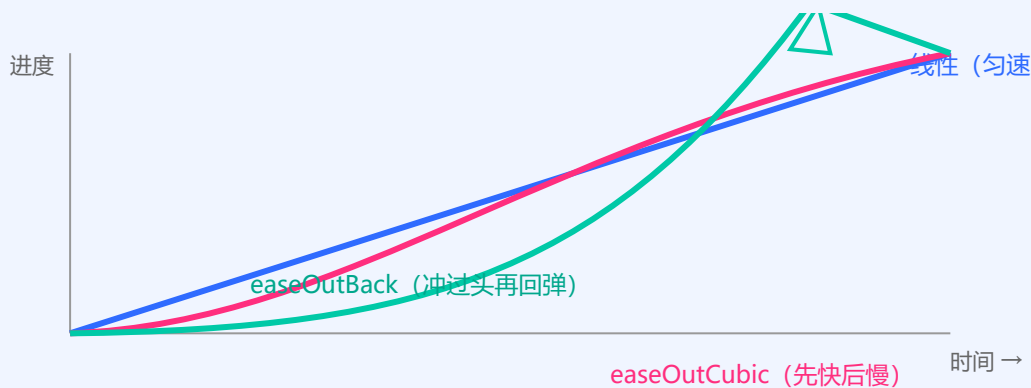


图 3：三种缓动曲线——回弹的那一截"过冲"就是果冻感的来源

5 图纸线条"重绘"动画的原理？

SVG 线条有一个鲜为人知的技巧：**stroke-dasharray** 本来是用来画"虚线"的（把实线切成一段一段），把虚线间隔设得比线还长，实线就"消失"了；再用 **stroke-dashoffset**（虚线图案整体偏移量）从大到小过渡，露出来的部分就越来越多——视觉上就是线条正在被画出来。

stroke-dasharray = 600

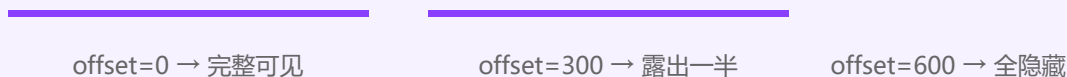


图 4：同一个 **stroke-dashoffset** 动画，配合 CSS transition 逐条线错峰延迟就是"现场画图"

复刻里给每条线加 **transition: stroke-dashoffset .9s ease (i*0.05)s**——第 i 条线延迟 $0.05 \times i$ 秒，于是 14 根轮廓线像排好队一样依次描出，这就是工程图"长出来"的机关。

6 图纸为什么也要用代码生成？

最容易的做法是找张坦克线稿贴上去——但那样图纸是“死”的。本复刻让**图纸和 3D 模型共享同一份零件数据**：每个模型定义里既有 3D 零件清单（几何体+坐标），也有 2D 图纸规格（矩形/圆/线的坐标，单位同为“米”）。画图纸就是把 2D 坐标 $\times 40$ 变成像素，翻译成 `<rect>` `<circle>` `<line>`。

好处立竿见影：**换模型时图纸跟着自动换**（切到机器人，左侧立刻变成机器人图纸）；改一个零件尺寸，图纸和 3D 同步变。这也正是推文作者说的下一步——“让 AI 生成准确的机器人设计图纸”：AI 只要输出结构化的零件清单，图纸和模型一次全有。这个“一份数据、两种投影”的思路和知识库 [[双时钟模型]] 异曲同工：**事实只存一份，视图只是投影**。

7 尺寸标注线（工程图味儿）怎么画？

工程图纸的“专业感”很大程度来自尺寸标注。它不是一根线，而是**四件套**：两端超出轮廓一小截的 **tick 短线** + 中间的**标注线** + 两端的**箭头**（SVG marker，画一次三角箭头复用）+ 居中的**文字**（如 “4200 mm”）。



图 5：一条水平尺寸标注 = tick + 线 + 双向箭头 + 文字；垂直标注把四件套转 90°

本复刻把它写成 `{t:'dim', x1, y1, x2, y2, label}` 数据，SVG 生成器统一处理——又一份“数据即图纸”的例证。

8 炮塔转动时，炮管为什么自动跟着转？

这是知识库概念 **Three.js 场景图**（父子层级树）的直接应用：炮塔底座、炮管、天线没有直接挂到场景里，而是先挂在一个 `turretNode` 空节点下，`turretNode` 再挂到坦克上、放在车体顶面。之后每帧只转 `turretNode`：**父动，子随动**——炮管天线不用写一行自己的旋转代码。



图 6：场景图父子层级——只转父节点 `turretNode`，三个子零件整体跟着转

这也是“模型可替换”的前提：层级本身就是一种建模（见 [[Three.js与场景图]]），坦克换成机器人时只是换一棵树。

9 “可替换为其他模型”架构上怎么设计？

核心就一句话：**把模型变成数据，别写成流程**。本复刻定义 `MODELS = { tank: {...}, robot: {...} }`，每个模型只有两样东西：`parts3d` 零件清单（几何体工厂+颜色+位置+挂在哪个父节点）和 `bp2d` 图纸规格。构建器 `buildModel(key)` 对任何模型走同一条流水线：清单 → 几何体 → 场景图。

想加一台挖掘机？只要再写一个 `excavator` 条目——**页面其他代码一行不用动**。下拉框切换时调 `resetAll(key)` 即完成“整体替换”。推文的 “Super easy to swap it out” 落地后就是这个形状。

10 为什么必须用 http 打开、双击 html 不行？

复刻页面用了 `<script type="module">`（ES module）。浏览器安全策略规定：**file:// 协议下模块脚本一律被 CORS 拦截**——双击打开只会看到一片空白（控制台报 CORS 错误）。

解法 trivial：`python -m http.server 8977` 一行起本地服务器，访问 `http://127.0.0.1:8977` 即可。知识库 [[零依赖编程]] 的 Python 标准库再次救场，不用装 Node。

11 Kimi K3 在这类 vibe coding 里干什么？

Kimi K3 是月之暗面 2026 年的旗舰代码模型（MoE 架构、2.8T 参数、1M 上下文、多模态输入）。"Vibe coding"（氛围编程）指人用自然语言描述想要的应用，AI 生成全部代码，人只负责看效果、提下一轮修改。在这条推文的场景里，K3 干的就是**生成整个 Three.js 应用**：场景搭建、程序化坦克、过渡动画。

值得注意的"咬合点"：**模型生成的东西越容易验证，AI 编程越靠谱**——网页应用天然"跑一下就知道对不对"（对比：让它写一个没有测试的底层库）。本指南的复刻实验正是把这个验证环节做成无头截图（Q12），形成"AI 生成 → 自动验证"的闭环。

12 没有显示器也能验证 3D 页面跑通了？

能。Edge/Chromium 的无头模式内置软件渲染（SwiftShader），没有显卡也能跑 WebGL：`msedge -headless=new --disable-gpu --screenshot= out.png --virtual-time-budget=4000 "http://127.0.0.1:8977/?state=3d"` 一行即把渲染结果存成 PNG。`--virtual-time-budget` 让页面"快进"4 秒虚拟时间，等动画/异步加载都结束再截图。

复刻页面还内置了测试钩子：右下角输出 `PARTS=14 DRAWCALLS=15`（零件数与绘制调用数），截图里直接可读——**零件数对不对 = 模型生成对不对**，零人工检查。本实验三张截图（图纸态/坦克/机器人）全部这样产出。

这个项目给我什么（能力清单）

- **复刻"2D→3D 自然过渡"**：图纸重绘 + 零件错峰挤出 + 缓动回弹，三件套拼出高级感过渡；
- **程序化建模实战**：14 个零件的坦克 + 8 个零件的机器人，全部代码生成，图纸与模型共享一份数据；
- **SVG 工程图纸生成**：含网格、尺寸标注、箭头的"真·图纸风"由数据驱动生成，可套用到任何项目；
- **场景图层级思维**：turretNode 空节点示范"父动子随"，是后续一切复杂 3D 的组织方式；
- **无头验证 WebGL**：一条 Edge 命令截图 + 页面内置统计钩子，让"AI 生成的页面"可被自动验收；
- **vibe coding 的质量闭环**：AI 生成代码 → 本地起服务 → 无头截图比对，适合所有 AI 生成的前端产物。

动手实验：30 分钟复刻"图纸变坦克"

实验材料在 `blueprint-to-3d/exercise/` (`index.html` + `app.js` + `vendor/` 本地化 Three.js) 。全部命令已在本机实测通过。

步骤 1 • 准备本地 Three.js (离线可跑)

```
cd F:\reminder\blueprint-to-3d\exercise\vendor
curl -sL -o three.module.js https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.module.js
curl -sL -o OrbitControls.js https://cdn.jsdelivr.net/npm/three@0.160.0/examples/jsm/controls/OrbitControls.js
```

目的：Three.js 下载到本地，页面不依赖外网。验证：`vendor\three.module.js` 约 1.27 MB，开头含 `REVISION = '160'`。

✅ 实测：`three.module.js` 1,272,972 字节，`OrbitControls.js` 1417 行 (`import ... from 'three'`)

步骤 2 • 起本地服务 (ES module 不能双击 `file://` 打开)

```
cd F:\reminder\blueprint-to-3d\exercise
python -m http.server 8977 --bind 127.0.0.1
```

目的：绕过 `file://` 的模块 CORS 限制。验证：

```
curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:8977/index.html
```

✅ 实测输出：200 server ok

步骤 3 • 打开页面看"图纸态"

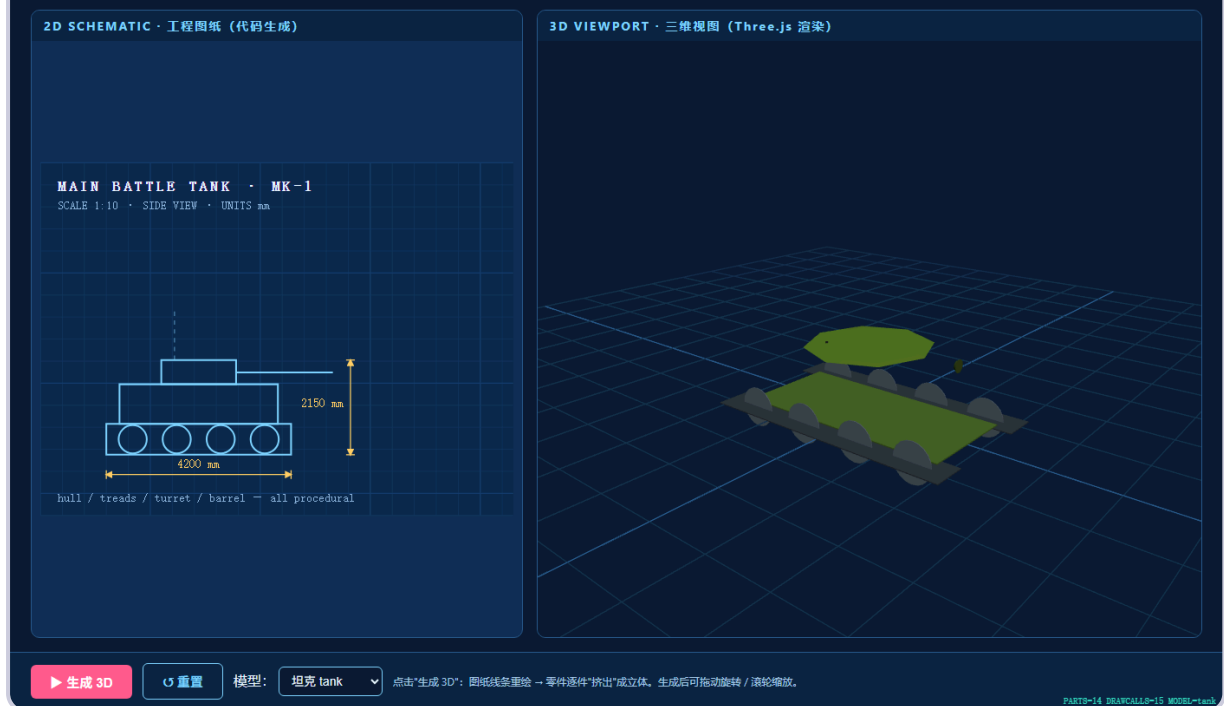
浏览器访问 `http://127.0.0.1:8977/index.html`

目的：左侧是由数据生成的坦克工程图纸（网格+尺寸标注），右侧 3D 区零件全部压扁成"纸片"待命。验证标准：

✅ 预期结果：图纸含 "MAIN BATTLE TANK · MK-1" 标题、4200 mm / 2150 mm 两道黄色尺寸标注；右下角绿色统计 `PARTS=14 DRAWCALLS=15`。本实验无头截图 `01_blueprint.png` 即此状态。

2D 工程图纸 → 3D 模型 · 自然过渡复刻

灵感来源：Dilum Sanjaya 用 Kimi K3 制作的 2D→3D 过渡应用（坦克完全由代码生成，可替换为其他模型），本页为 Three.js 手工复刻学习版。



步骤 4 · 点"生成 3D"看过渡

目的：亲眼看"自然过渡"——图纸线条逐条描出、淡出，右侧零件像果冻一样逐件挤出。约 2 秒后动画结束，鼠标可拖动旋转、滚轮缩放，炮塔持续缓慢自转（场景图父子随）。验证标准：

✅ **预期结果：**能看到履带→轮子→车体→炮塔→炮管依次"长"出并带轻微回弹；拖动能 360° 旋转坦克。

步骤 5 · 换成机器人，验证"模型可替换"

页面底部下拉框选择 "机器人 robot"

目的：MODELS 数据驱动——不换代码只换数据，左侧图纸同步变成机器人图纸（SERVICE ROBOT · R-2，2000/3100 mm 标注）。再点一次"生成 3D"。验证标准：

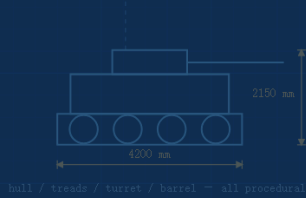
✅ **预期结果：**统计变为 **PARTS=8**，3D 区出现白身红臂蓝腿的机器人。本实验无头截图 03_robot_3d.png 即此状态。

2D 工程图纸 → 3D 模型 · 自然过渡复刻

灵感来源: Dilum Sanjaya 用 Kimi K3 制作的 2D→3D 过渡应用 (坦克完全由代码生成, 可替换为其他模型), 本页为 Three.js 手工复刻学习版。

2D SCHEMATIC · 工程图纸 (代码生成)

MAIN BATTLE TANK · MK-1
SCALE 1:10 · SIDE VIEW · UNITS mm



▶ 生成 3D

🔄 重置

模型:

坦克 tank

点击“生成 3D”: 图纸线条重绘 → 零件逐件“拼出”成立体。生成后可拖动旋转 / 滚轮缩放。

PARTS=14 DRAWCALLS=15 MODEL=tank

3D VIEWPORT · 三维视图 (Three.js 渲染)

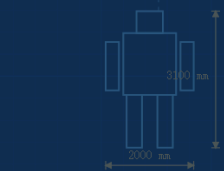


2D 工程图纸 → 3D 模型 · 自然过渡复刻

灵感来源: Dilum Sanjaya 用 Kimi K3 制作的 2D→3D 过渡应用 (坦克完全由代码生成, 可替换为其他模型), 本页为 Three.js 手工复刻学习版。

2D SCHEMATIC · 工程图纸 (代码生成)

SERVICE ROBOT · R-2
SCALE 1:10 · SIDE VIEW · UNITS mm



▶ 生成 3D

🔄 重置

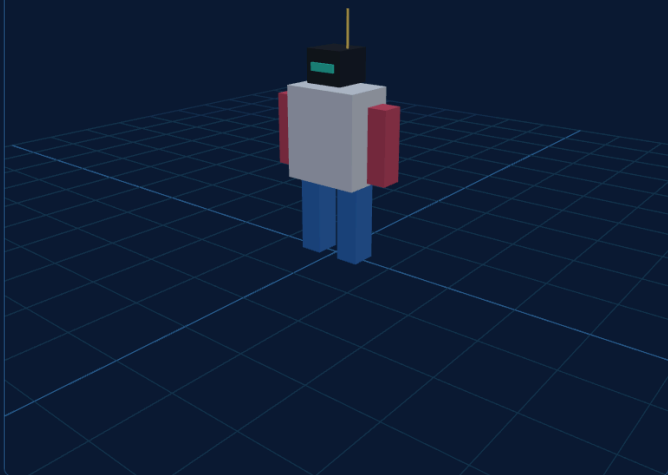
模型:

机器人 robot

点击“生成 3D”: 图纸线条重绘 → 零件逐件“拼出”成立体。生成后可拖动旋转 / 滚轮缩放。

PARTS=8 DRAWCALLS=9 MODEL=robot

3D VIEWPORT · 三维视图 (Three.js 渲染)



步骤 6 · 无头截图自动验收 (可选进阶)

```
"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe" --headless=new --disable-gpu ^  
--window-size=1400,900 --virtual-time-budget=4000 ^
```

```
--screenshot="F:\reminder\blueprint-to-3d\exercise\shots\02_tank_3d.png" ^  
"http://127.0.0.1:8977/index.html?state=3d&model=tank"
```

目的：没有显示器也能验收 AI/自己写的 WebGL 页面；`?state=3d` 让页面跳过动画直达最终状态。验证标准：

✅ **实测输出：** `132296 bytes written to file ...02_tank_3d.png`；打开 PNG 可见完整 3D 坦克（本 PDF 中上图即该截图原图）。